# *The TRION*

## 1. Project Overview

The TRION is a 2D point-and-click escape room game. In the game you play as a lab assistant who lost their memory and was trapped in a research lab. Because of an experiment gone wrong, the room is stuck in three time periods at the same time (past, present, future). Your goal is to find the way out of this place.

## 2. Project Review

Cube Escape: In the game, players are trapped in a single room but must travel between four different time periods (Spring 1964, Summer 1971, Fall 1971, and Winter 1981).
Improvement plan to make:
In escape you have to interact with some object to change time but The TRION you can travel in time anywhere and the UI is related to the game lore.also in this game changing something in the past (like planting a seed) instantly updates the future environment.

## 3. Programming Development

### 3.1 Game Concept

Story :

You awaken as an amnesia lab assistant trapped in a research lab.A quantum experiment gone wrong and shattered time into 3 periods at the same time. You have a mysterious device called “Trion Tuner” and you can change the time period that you live in. To escape from this lab you have to solve the puzzle in this game and find a way out.

Mechanics :

Time shifting from click on top right corner to change time

Solve puzzle cause effect across time

Collect items from different areas to use on objects in the environment.

Selling Point :

Trying to solve puzzles not just in one room. You have to travel through time or zone to solve the full puzzle.

While solving puzzles you will slowly understand what happens in this place.

The interactive Time Slider on the screen is the actual sci-fi device your character is holding, deeply immersing the player in the lore.

## 3.2 Object-Oriented Programming Implementation

| Class Game : Central control class | | | |
|---|---|---|---|
| Attribute | | Method | |
| current_time | Store current state of time | run() | Start the game loop.Update state in the game/draw |
| current_scene | Store current scene that player is in | handle_event() | Handle player's input like mouse click or pressing keyboard |
| inventory | Store player's item in inventory | change_time( new_time) | Update new state of time and update scene |

| Class Scene: manage items and function in each scene of the game | | | |
|---|---|---|---|
| Attribute | | Method | |
| scene_id | Store id of each scene | draw(screen, current_time) | Draw background and items that in current time scene |
| background | Store path of each scene's picture | handle_click(mouse_position) | Check whether mouse position when click is collide with items in current scene or not |
| items | List of objects that is in the scene | check_hover(mouse_position) | Check whether mouse position is collide with items in current scene or not for changing mouse's cursor |

| Class Item: class for interact with each items in the game | | | |
|---|---|---|---|
| Attribute | | Method | |
| name | Item's name | draw(screen) | Draw item in game |
| images | Store items's images path | collect(inventory) | Function for collect item into player's inventory |
| rect | Hit box for check whether its collide with mouse or not | on_click() | Unique logic for each item when it get clicked |
| time_period | Assign items in each time period | | |
| is_collectable | Boolean true or false for collectable or not | | |

| Class Inventory: manage player's inventory | | | |
|---|---|---|---|
| Attribute | | Method | |
| slots | List of item that player is collect | add_item(item) | Add item into player's inventory |
| selected_item | Item that is in using | remove_item(item) | Remove item when player done with using it |
| ui_rect | Hit box of inventory slots | draw(screen) | Draw inventory UI on screen |

| Class TimeSlider: UI for changing time | | | |
|---|---|---|---|
| Attribute | | Method | |
| rect | Background of the time slider | draw(screen) | Draw time slider on screen |
| rect_knob | Moveable square | click_handle(mouse_position) | Handle when player click on it |
| segment_width | Divide slider into three button | | |

## 3.3 Algorithms Involved

| Algorithm | Logic |
|---|---|
| finite state machine | When players interact with the TimeSlider, a global variable updates.This is for scene class determine which version of scene to render |
| State mapping | Object in the future or present may programmed with mapping between each other on action across time example player uses item seed, set tree_plant = True for in present the tree will appear |
| AABB collision detection | Each object has a rectangular hit box.when mouse-click event occurs, the game runs a function to check all active items in current scene |

# 4. Statistical Data (Prop Stats)

## 4.1 Data Features

| Feature | Why is it good to have this data? What can it be used for? | How will you obtain 100 values of this feature data? | Which variable (and which class will you collect this from)? | How will you display this feature data (via summarization statistics or via graph)? |
|---|---|---|---|---|
| Click Coordinates | tracking where players click, we can see if they are exploring the whole room or only clicking in one corner. Helps verify if puzzle items are well distributed | every time the player clicks the mouse button anywhere on the screen | Pos_click in Game class | 2 bar graph |
| Clicks Per Dimension | If players click 50 times in the Past but only 2 times in the Future before switching back, the Future dimension might lack interactive | counting total mouse clicks between each use of the Time Slider. | Current_time in Game class | Boxplot |

| | | | | |
|---|---|---|---|---|
| | content or clues. | | | |
| Clicks Between each Puzzle | If player click on puzzle 2 more than 3 this can conclude puzzle 2 is harder than 3 | Counting each mouse clicks between each puzzle | Current puzzle state in Game class | Table of Statistical Values |
| Thinking time | calculating the time delta between two consecutive clicks. If the elapsed time is > 3 seconds, the game logs one single row recording that exact duration. | Counting every 2 seconds whether player is in thinking state or not | idle_duration in Game class | Line graph |
| Zone entrance count | If there is low entrance count on that zone therefore that zone has not well distribute puzzle | Count every time that players enter each zone | From Game class | Bar graph |

| | Feature name | Graph objective | Graph type | X-axis | Y-axis |
|---|---|---|---|---|---|
| Graph1 | Click Coordinates (distribution) | visualize coordinates of player clicks to see which areas of the room are explored the most | 2 bar graph | Screen x coordinate | Screen y coordinate |
| Graph2 | Clicks Per Dimension | To compare the distribution of interaction between the Past, Present, and Future dimensions | Boxplot | Dimension | Number of clicks |
| Graph3 | Thinking time | To track how thinking time spikes over the gameplay. | Line graph | Gameplay Timeline | Idle Duration |

| | | | | | |
|---|---|---|---|---|---|
| Graph4 | Zone entrance count | To compare how often players visit and backtrack to different zones to evaluate puzzle distribution. | Bar graph | Zone name | Entrance count |

## 4.2 Data Recording Method

The data stored in a CSV fill.Every time a tracked event occurs such as a coordinate click, a time-dimension shift, or a puzzle completion the game will append a new row containing that session along with the specific feature data.

## 4.3 Data Analysis Report

With pandas library in python.And evaluate mean median SD And Present using matplotlib and seaborn with

- 2 Bar graph - click coordinate
- Boxplot - compare click in each dimension
- Histogram - distribution click between each puzzle
- Line graph - track thinking time frequency
- Bar graph - compare zone enter count

## 5. Project Planning Timeline

| Week | Week | Task |
|---|---|---|

| | | |
|---|---|---|
| 1 | 8 | Proposal submission / Project initiation |
| 2 | 9 | Implement basic class and scene |
| 3 | 10 | Implement time travel and interaction |
| 4 | 11 | Inventory system and items |
| 5 | 12 | Design and put puzzle in the game |
| 6 | 13 | Game test and bug fixing |
| 7 | 14 | Last check Submission week |

## 6. Document version

Version: 1.0
Date: 4 April 2026
Editor: Anawin Chaichana

| Date | Name | Feedback |
|---|---|---|
| 16/03 | Peraya | **Major Changes Needed**<br><br>- **Section 3.3:** You don't provide a technical algorithm. Listing "Event-Driven Architecture" or "Clicking on objects" describes a design pattern or user input, not an algorithm.<br>- **Section 4.1:** You are at extreme risk of failing the 100 records per feature rule. Your current plan to log data "every 5 seconds" is a time-series snapshot, not a gameplay event. If a player gets stuck on a puzzle and stands still for 10 minutes, you will have 120 rows of identical, useless data. You must redefine these to be Action-Based. A player will easily click 100 times, but they might not change their inventory count 100 times.<br>- **Presentation Format (Section 4.3):** Your description is too vague . Simply saying "Line graph and plot graph" does not explain the *relationship* you are visualizing. You need to specify exactly what the axes represent to prove your data is useful.<br><br>**Minor Changes Needed**<br><br>- **Data Recording Method (Section 4.2):** While JSON is acceptable, ensure your DataLogger class is capable of |

| | | |
|---|---|---|
| | | handling high-frequency writes without causing "lag" during the point-and-click interactions.<br>- **Analysis Depth (Section 4.3):** Your analysis of Averages is too surface-level. You should explain the correlation for example, does a high “Click Frequency” in a specific room correlate with a longer “Time to Solve,” indicating a poorly designed or confusing puzzle? |
| 30/03 | Peraya | - You have a Graph Plan and a Feature Table, but you are missing the Table of Statistical Values (showing Mean, Median, Max, Min, etc.). The rubric requires at least 1 table specifically for showing statistical values.<br>- You defined Thinking time as Counting every 2 seconds. This is a background timer, not a feature or event. If the player sits still for 200 seconds, you get 100 identical rows. This provides zero statistical value. You should change this to Time taken per puzzle or Duration of inactivity before a click. |
| 04/04 | Peraya | - **Graph Consistency:** In section 4.1, you list "Graph 1" for Click Coordinates as a **Scatter Plot**, but in section 4.3 and your graph table, you refer to it as a **Heatmap**. These are different visualization types; the proposal must be consistent.<br>- **Graph type mismatch:** You should not use Scatter plot or heatmap for Click coordinates. You should use 2 bar graph or 3d bar graph |